

CK3 Character History Generator

Master Technical Specification & Implementation Guide

Target Architecture: Full-Stack Cloud Web Application

Prepared for: Downstream LLM Implementation (Claude Max)

Implementation Mandate

This document contains the strict, exhaustive technical requirements for building the web-based simulation engine. It replaces all previous code-heavy drafts with deep, descriptive semantic instructions. The interpreting LLM (Claude Max) is expected to synthesize this architectural intent into optimized React/Python code.

CRITICAL DIRECTIVES: 1. The output formats defined in Chapter 9 are IMMUTABLE constraints matching the Clausewitz engine. Do not alter their syntax. 2. Features explicitly omitted (e.g., culling, cadet branches, dynamic nicknames, 3D DNA strings) MUST NOT be implemented to preserve project scope and server performance.

May 1, 2026

Contents

1	Project Overview & Cloud Architecture	3
1.1	1.1 Project Objective	3
1.2	1.2 Transition to Cloud Web Architecture	3
2	UI/UX Design System & Philosophy	4
2.1	2.1 Monochrome Modernism	4
2.2	2.2 Layout & Workspace Topology	4
2.2.1	2.2.1 The Left Sidebar (Global Navigation & File I/O)	4
2.2.2	2.2.2 The Center Workspace (Contextual Editor)	5
2.2.3	2.2.3 The Right Drawer (Logs & Visualizations)	5
3	Frontend State Management (Redux/Zustand)	6
3.1	3.1 User-Defined Variables (The State Object)	6
3.2	3.2 Hardcoded Engine Logic	6
4	Backend Data Dictionary & Schemas	8
4.1	4.1 The Character Entity	8
4.2	4.2 The Genetic Trait Entity	8
4.3	4.3 The Death Reason Entity	9
4.4	4.4 The Title Entity	9
5	The Parsing Subsystem (AST Compiler)	10
5.1	5.1 The Core Tokenization Algorithm	10
5.2	5.2 The Recursive Parsing Algorithm	10
5.3	5.3 Transforming the AST into Title Hierarchies	11
5.4	5.4 Extracting Genetics and Deaths	11
6	Simulation Mechanics: Genetics & Mortality	12
6.1	6.1 The Genetics Engine Algorithm	12
6.2	6.2 Mortality, Killers, and Executions	12
6.2.1	6.2.1 Global Mortality Spikes	12
6.2.2	6.2.2 Natural Deaths	13
6.2.3	6.2.3 Hostile Deaths (Explicit Killers)	13
7	Simulation Mechanics: Title Transitions	14
7.1	7.1 Peaceful Transition: Marriage	14
7.2	7.2 Hostile Transition: Usurpation & The Claimant System	14
7.3	7.3 Transition: Extinction	15
8	Frontend User Experience: Title Sequences	16
8.1	8.1 The Global Timeline View	16

8.2	8.2 Interaction Mechanics	16
9	Immutable Output Formatting (Paradox Script)	17
9.1	9.1 Character History Format (<code>character_history.txt</code>)	17
9.2	9.2 Title History Format (<code>title_history.txt</code>)	18

Chapter 1

Project Overview & Cloud Architecture

1.1 Project Objective

The CK3 Character History Generator procedurally creates deep, multi-generational character and title histories for *Crusader Kings III* total conversion mods. The tool accelerates mod development by bypassing manual script-writing, simulating centuries of life, death, and succession, and outputting raw Paradox-formatted text files ready for game integration.

1.2 Transition to Cloud Web Architecture

The application must be designed as a scalable, cloud-hosted web service. This fundamentally changes how files, state, and processing are handled compared to a local desktop app.

- **File Handling:** The system cannot read from the user's local hard drive. The frontend must implement drag-and-drop file upload zones. Uploaded files must be securely transmitted to the backend, temporarily stored in a cloud bucket (e.g., AWS S3) or temporary server storage, processed during the simulation, and subsequently purged.
- **Asynchronous Processing:** Simulating thousands of characters over thousands of years is a blocking, CPU-bound operation. If attempted synchronously, the browser will encounter an HTTP timeout. Therefore, the architecture **must** utilize a Task Queue pattern. The backend framework (FastAPI) will receive the configuration payload and dispatch it to a background worker (Celery) via a message broker (Redis). The frontend will receive a task ID and poll a status endpoint until the generation is complete.
- **Client-Side Visualization:** Local command-line binaries (like Graphviz `dot.exe`) are no longer accessible. Any visual family tree generation must be accomplished on the client side using the JSON data returned by the backend. The frontend should leverage modern web-native SVG libraries such as D3.js or WebAssembly ports of Graphviz (Viz.js).

Chapter 2

UI/UX Design System & Philosophy

2.1 2.1 Monochrome Modernism

The frontend must be built using React (Next.js or Vite) and Tailwind CSS. The design language must be strictly monochrome to ensure a professional, utilitarian, "Zero-Friction" experience. The LLM generating the frontend code should restrict its Tailwind palette strictly to grayscales.

- **Backgrounds:** Pure white (`bg-white`) for the main canvas, and very light gray (`bg-gray-50` or `bg-gray-100`) for interactive cards, sidebars, and input fields.
- **Text & Borders:** Pure black (`text-black`) for primary headings and active states. Medium-to-dark grays (`text-gray-600`, `border-gray-300`) for secondary text, labels, and panel dividers.
- **Typography:** Sans-serif system fonts (Inter or Roboto). Visual hierarchy must be established exclusively through font size (`text-sm`, `text-2xl`) and weight (`font-light`, `font-extrabold`), avoiding colored text entirely.

2.2 2.2 Layout & Workspace Topology

The user interface should be a Single-Page Application (SPA) divided into three distinct vertical columns to maximize horizontal screen real estate for deep data entry.

2.2.1 The Left Sidebar (Global Navigation & File I/O)

A fixed-width (e.g., 250px) panel anchored to the left of the screen. It contains:

1. **Dropzones:** Clean, dashed-border areas where users can drop their Paradox `.txt` files (Titles, Traits, Death Reasons, Name Lists). When a file is uploaded, the UI should show a success checkmark and the filename.
2. **Navigation:** Simple, bold text links to switch the center workspace between "Global Settings", "Life Cycle Modifiers", "Negative Events", and "Title Histories".
3. **Execution Button:** A large, dark-gray button anchored at the bottom labeled "Generate Simulation". This button must remain disabled (grayed out and unclickable) until at least the Titles and Name List files have been successfully parsed and loaded into the global state.

2.2.2 The Center Workspace (Contextual Editor)

A fluid-width container that changes based on the navigation selection. Form inputs here should not have individual "Save" buttons. Instead, utilizing React state and Redux, every change to an input field or slider should instantly update the global state store (using `onBlur` or debounced `onChange` events).

2.2.3 The Right Drawer (Logs & Visualizations)

By default, this panel is hidden. When the user clicks "Generate Simulation", this drawer slides out from the right side of the screen (taking up roughly 30% of the viewport). It displays a mocked "terminal" window streaming the polling status of the Celery task (e.g., "Parsing titles...", "Simulating year 6850...", "Packaging ZIP..."). Once the simulation completes, this drawer provides the final Download Button and a button to view the generated family trees.

Chapter 3

Frontend State Management (Redux/Zustand)

The frontend must maintain a single, highly structured Global State object. This object acts as the ultimate source of truth. When the user triggers the simulation, this entire object is serialized into JSON and sent to the backend.

3.1 3.1 User-Defined Variables (The State Object)

The global state must track the following distinct categories of data:

Global Settings Integer values defining the bounds of the simulation: `start_year`, `end_year`, and `maximum_generations`.

Life Cycle Modifiers Integer and float values controlling demographic probabilities: `max_age_difference`, `max_children_per_couple`, `base_fertility_rate`, `male_bastard_chance`, and `female_bastard_chance`.

Negative Events An array of objects representing global mortality spikes (e.g., plagues or wars). Each object must contain a unique UUID, a string `name`, an integer `start_year`, an integer `end_year`, and a float `mortality_multiplier` (e.g., 5.0 means characters are 5x more likely to die during these years).

Parsed File Data Dictionaries holding the data returned from the backend after the user uploads their text files. This includes the parsed AST of the map titles, the validated genetic traits, the valid death reasons, and arrays of culturally specific names.

Title Sequences The most complex state object. A dictionary where the key is a specific map Title ID (e.g., "k_gondor"), and the value is an ordered array of Sequence Objects. Each Sequence Object represents a dynasty's rule and contains: the string ID of the Dynasty, the duration type (either "years" or "generations"), the integer duration value, and the transition method ("marriage", "usurpation", or "extinction").

3.2 3.2 Hardcoded Engine Logic

The frontend developer (LLM) must understand what is **not** in the global state. The mathematical formulas that calculate exact mortality curves based on age, the recursive logic that cascades inheritance down a title hierarchy, and the final string formatting of the output files are hardcoded into the backend Python engine. The user configures the weights and the timeline; the engine

handles the literal simulation.

Chapter 4

Backend Data Dictionary & Schemas

The backend API (FastAPI) will receive the frontend's global state payload. To ensure data integrity, the backend must define strict Pydantic schemas (or Python Dataclasses) for all entities.

4.1 4.1 The Character Entity

The Character object is the atomic unit of the simulation loop. The schema must require the following fields:

- **Identity:** A generated string ID, a string Name, a string Dynasty House, a string Religion, a string Culture, and a boolean `is_female` flag.
- **Relationships:** Optional string IDs for Father and Mother. Lists of string IDs for Spouses and Children.
- **Attributes:** A list of strings representing the character's Genetic and Personality traits.
- **Chronology:** A string representing the exact Birth Date (formatted YYYY.M.D). An optional string for Death Date.
- **Mortality Mechanics:** A string representing the Death Reason ID (which defaults to a generic natural causes ID), and an optional string representing the Killer ID (used if the character was murdered or executed).
- **Employment Mechanics:** Optional strings for an Employer ID and the Date of Employment. This is critical for tracking displaced rulers (Claimants).

4.2 4.2 The Genetic Trait Entity

Extracted from the user's uploaded trait files. The schema must include:

- A string ID, a string Group (used to group tiered traits together), and an integer Level.
- **Probabilities:** Floats for `birth_chance` and `random_creation_chance`.
- **Constraints:** A list of opposite trait IDs (traits that cannot logically coexist with this one).

4.3 4.3 The Death Reason Entity

Extracted from the user's uploaded death reason files. The schema must include:

- A string ID, a boolean flag indicating if the death is considered "natural", and an optional string representing a required trait trigger (e.g., a character must have the "sickly" trait to roll this death reason).

4.4 4.4 The Title Entity

Extracted from the user's uploaded landed titles file. The schema must include:

- A string ID, a string indicating the Tier (hegemony, empire, kingdom, duchy, county, barony), and a boolean `is_landed` flag.
- A dictionary containing raw metadata (color arrays, capital IDs, cultural names).
- A recursive dictionary containing child Title Entities (representing the *de jure* geographical hierarchy).

Chapter 5

The Parsing Subsystem (AST Compiler)

The engine must read proprietary Paradox Interactive `.txt` files. Because these files mix key-value pairs with nested brackets, lack quotation marks around strings, and allow duplicate keys within the same block, a standard JSON parser will instantly crash. The backend developer must write a custom Abstract Syntax Tree (AST) parser in Python.

5.1 5.1 The Core Tokenization Algorithm

Before parsing structure, the raw text file must be sanitized and tokenized. The algorithm must:

1. Use a regular expression to find and remove all comments (any text following a `#` symbol until the end of the line).
2. Pad all structural characters—specifically opening braces, closing braces, and equals signs—with whitespace on both sides. This isolates them as distinct tokens.
3. Split the resulting string by whitespace to create a list of tokens. The split logic must utilize regex to ensure that substrings enclosed in double quotes are preserved as single, unbroken tokens.

5.2 5.2 The Recursive Parsing Algorithm

The backend must iterate through the list of tokens using a recursive function that maintains a state stack.

- The parser looks for a string token immediately followed by an equals sign and an opening brace. This signals the start of a nested dictionary block, prompting a recursive call.
- **The Duplicate Key Edge Case:** If the parser encounters a key-value assignment (e.g., `trait = brave`), it must check if the key `trait` already exists in the current dictionary scope. If it does, the parser must convert the existing string value into a Python list and append the new value, rather than overwriting the key.

5.3 5.3 Transforming the AST into Title Hierarchies

Once the basic AST dictionary is built, the engine must iterate through it to establish the geographical map hierarchy.

- **Entity Identification:** Any dictionary key that begins with the exact prefixes `h_`, `e_`, `k_`, `d_`, `c_`, or `b_` must be classified as a map title. All other keys (like `color` or `capital`) must be bundled into a nested `metadata` dictionary attached to the parent title.
- **The Hegemony Edge Case:** Hegemonies (prefix `h_`) are a unique tier above empires. In the game engine, they can be either titular (off-map, existing only in concept) or landed (physically existing on the map). The parser must determine this dynamically: if an `h_` block contains zero child keys matching standard map prefixes, the parser must flag the title as `is_landed: false`. If it contains child map entities, it must be flagged `is_landed: true`.

5.4 5.4 Extracting Genetics and Deaths

The AST parser is also used for the Traits and Death files.

- **Traits:** The engine must iterate over the parsed traits and discard any trait that does not contain the exact key-value pair `genetic = yes`. UI-specific data (like description localization strings) must be purged to save memory.
- **Deaths:** The engine must extract the `natural = yes` flag. It must also check for a nested `natural_death_trigger` block. If found, it extracts the `has_trait` value to map specific deaths to specific character afflictions.

Chapter 6

Simulation Mechanics: Genetics & Mortality

6.1 6.1 The Genetics Engine Algorithm

When the chronological simulation loop spawns a new child character, the engine must execute a genetics calculation based on the validated `GeneticTrait` registry.

1. **Parent Trait Evaluation:** The engine compiles a list of all genetic traits possessed by the mother and father. It cross-references these against the `opposites` array defined in the trait schema. If a child theoretically inherits two opposite traits, they cancel each other out and neither is applied.
2. **Active Inheritance (Homogenous):** If both parents possess a trait belonging to the same `group`, the child has a highly weighted probability (e.g., 80%) to inherit that trait at the highest `level` present in the parents. There is a smaller probability (e.g., 20%) that the trait mutates and is inherited at one level higher, capped at the maximum level defined for that group in the registry.
3. **Passive Inheritance (Heterogenous):** If only one parent possesses a trait, the child has a moderate probability (e.g., 50%) to inherit it at one level lower than the parent, and a minor probability (e.g., 10%) to inherit it at the exact same level.
4. **Spontaneous Mutation:** For all traits in the registry that neither parent possesses, the engine rolls a random number against the trait's parsed `random_creation` float value to determine if the trait spontaneously generates in the child.

6.2 6.2 Mortality, Killers, and Executions

During the annual tick of the simulation, characters undergo mortality checks. The base probability of death starts extremely low and scales exponentially as the character ages.

6.2.1 Global Mortality Spikes

If the current year of the simulation loop falls within the Start and End bounds of a user-defined "Negative Event", the engine multiplies the base mortality probability of all living characters by the event's user-defined `mortality_multiplier`.

6.2.2 Natural Deaths

If the RNG determines a character dies of natural causes (age, illness, childbirth), the engine filters the Death Reason registry for items where `is_natural` is true. If the dying character possesses a trait that matches a death reason's `required_trait` trigger, that specific death ID is heavily weighted. Otherwise, a random generic natural death is assigned.

6.2.3 Hostile Deaths (Explicit Killers)

During a hostile dynastic transition (detailed in Chapter 7), the reigning ruler is forcefully deposed.

1. The engine forces a mortality event on the outgoing ruler.
2. The engine selects a non-natural death reason from the registry (e.g., execution, murder in battle).
3. The engine identifies the antagonist—specifically, the primary character of the incoming Dynasty taking over the title.
4. The antagonist's character ID is explicitly assigned to the dying character's `killer_id` property. This ensures the game engine properly registers the blood feud.

Chapter 7

Simulation Mechanics: Title Transitions

The core utility of the application is mapping the succession of titles between different houses over centuries. The user defines these sequences in the UI. The simulation engine executes them chronologically.

The Cascading Inheritance Rule

If a succession sequence is assigned to a high-tier title (e.g., an Empire), the engine must automatically propagate that exact sequence to all child titles (Kingdoms, Duchies, Counties) nested underneath it in the parsed AST. The only exception is if the user has explicitly configured a different sequence for a specific sub-title in the UI, overriding the cascade.

7.1 7.1 Peaceful Transition: Marriage

- **Trigger Concept:** House A's predefined duration (measured in years or generations) expires. The UI dictated the transition should be "Marriage".
- **Engine Action:** The engine forces the final ruler of House A to generate a valid heir. Concurrently, the engine searches the incoming House B for an eligible, living character of the opposite sex. The engine forces a marriage event between these two characters.
- **Engine Result:** The engine explicitly overrides standard game rules regarding patrilineal/matrilineal marriages. It forces the resulting child of this union to be born exclusively into House B. When the ruler of House A dies naturally, the child inherits the title under the banner of House B.

7.2 7.2 Hostile Transition: Usurpation & The Claimant System

- **Trigger Concept:** House A's duration expires. The UI dictated the transition should be "Usurpation".
- **Engine Action:** The engine triggers an immediate, hostile mortality event (see Section 6.2) for the current ruler of House A. It immediately grants the title to the reigning head

of House B on that exact date.

- **The Claimant/Employer Edge Case:** If the usurped ruler of House A had immediate family members, they do not vanish. The engine evaluates them. If they hold no other landed titles, they are stripped of their noble status. The engine then scans the `WorldState` to find a different, active landed ruler who shares the displaced family's culture or faith. The engine assigns the displaced characters an `employer_id` pointing to that friendly ruler, and an `employer_date` matching the exact date of the usurpation. The characters flee to that court and persist in the background as claimants.

7.3 7.3 Transition: Extinction

- **Trigger Concept:** House A's duration expires. The UI dictated the transition should be "Extinction".
- **Engine Action:** The engine artificially sets the fertility multiplier of the final generation of House A to 0.0. The characters live out the remainder of their natural lifespans but produce no heirs.
- **Engine Result:** Upon the death of the final ruler, the engine generates a fabricated "distant claim" event, transferring the title to the head of House B.

Chapter 8

Frontend User Experience: Title Sequences

To prevent overwhelming the user when managing complex, multi-dynasty title histories, the web frontend requires a highly visual, interactive interface within the "Title Histories" workspace. The implementation must utilize an **Interactive Gantt Chart** paradigm.

8.1 8.1 The Global Timeline View

The interface presents a massive timeline emphasizing chronological overlap across multiple titles.

- **Y-Axis (The Hierarchy):** Represents the geographical tree of the parsed titles. The user can expand and collapse tiers (e.g., expanding the Empire of Blue Mountains reveals the Kingdom of Nogrod).
- **X-Axis (The Chronology):** A continuous, scrollable horizontal timeline representing the Simulation Years (from the global Start Year to End Year).

8.2 8.2 Interaction Mechanics

- **Sequence Creation:** Users drag and drop a "Dynasty Block" onto the timeline grid aligned with a specific title on the Y-Axis.
- **Duration Sizing:** The user clicks and drags the left or right edges of the Dynasty Block to visually resize its duration. The UI automatically calculates and displays the duration in generations/years inside the block.
- **Transition Configuration:** When the user snaps a new Dynasty Block directly against the end of an existing block, the UI generates a clickable visual "node" or "boundary line" between them. Clicking this boundary opens a minimalist popover allowing the user to select the Transition Type (Marriage, Usurpation, Extinction) via a simple dropdown menu.

Chapter 9

Immutable Output Formatting (Paradox Script)

The final phase of the Celery worker task is converting the Python `WorldState` into physical text files packaged in a ZIP archive.

Strict Formatting Requirement

The following syntax definitions for Character History and Title History are IMMUTABLE constraints defined by the Clausewitz game engine. The final text files generated by the backend MUST match these formats exactly, including specific key names, indentation styles, and chronological date blocks. The LLM writing the file-generation code must adhere to these templates flawlessly.

9.1 9.1 Character History Format (`character_history.txt`)

The engine iterates over all generated Character models. It generates a text block for each character formatted exactly as follows. Notice the conditional inclusion of the `killer`, `female`, and `employer` fields based on the character's life events.

```
1 random_character_id_1 = {
2     name = Character_Name
3     dynasty_house = house_name
4     religion = faith_name
5     culture = culture_name
6
7     female = yes # Must be omitted entirely if the character is male
8
9     father = father_id
10    mother = mother_id
11
12    trait = trait_name_1
13    trait = trait_name_2
14
15    YYYY.M.D = {
16        birth = yes
17    }
18
19    # Generated ONLY if the character was displaced via Usurpation
20    YYYY.M.D = {
```

```

21     employer = friendly_ruler_id
22 }
23
24 YYYY.M.D = {
25     death = {
26         death_reason = death_execution
27         killer = hostile_ruler_id  # Must be omitted if death is
                                   natural
28     }
29 }
30 }

```

9.2 9.2 Title History Format (title_history.txt)

The engine maps every map title ID to its chronological list of holders. Dates must be sequential. If a transition is peaceful, the transfer dates will align with the natural deaths of the rulers. If the transition is hostile, the dates will align with the exact usurpation event date.

```

1 k_gondor = {
2     6800.1.1 = {
3         holder = house_anarion_ruler_1
4     }
5     6840.5.12 = {
6         holder = house_anarion_ruler_2
7     }
8     6890.10.1 = {
9         # Hostile transition point / Usurpation
10        holder = house_telcontar_ruler_1
11    }
12 }

```